MediCare Technical Documentation

System Architecture, Request Handling, and Information Retrieval

Ahmad Mourad

1. Introduction

MediCare is a web application for browsing specialist clinics, booking appointments, checking medicine prices, placing demo pharmacy orders, and asking general health questions. It is built with Python, FastAPI, and scikit-learn on the backend, and standard HTML, CSS, and JavaScript modules on the frontend.

Instead of requiring users to fill out separate forms by hand, the website includes a chat assistant that reads plain English requests, extracts the needed details, and performs the matching action on the page:

- When a user asks to book a clinic or buy a medicine, the browser opens the correct modal window and fills in the fields automatically.
- When a user asks a general health question, the server searches a local Markdown knowledge base and fetches a short topic summary from the MedlinePlus XML service.

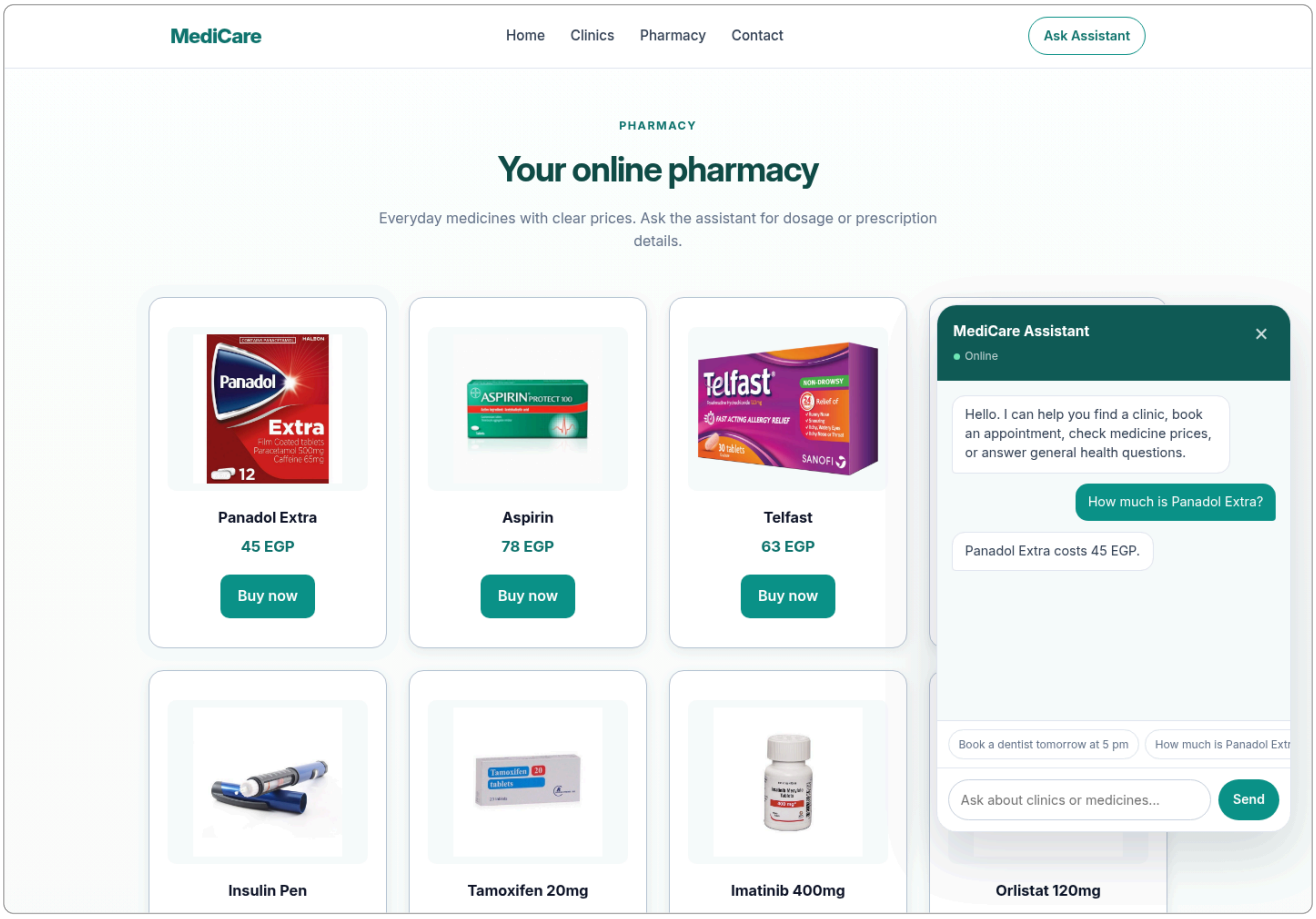


Figure 1: Hero landing section with the MediCare chat assistant open and answering a medicine price query.

2. System Architecture and How It Works

The application runs as a single FastAPI web server. It serves the static files in `Frontend/` and provides five JSON API routes under `/api`.

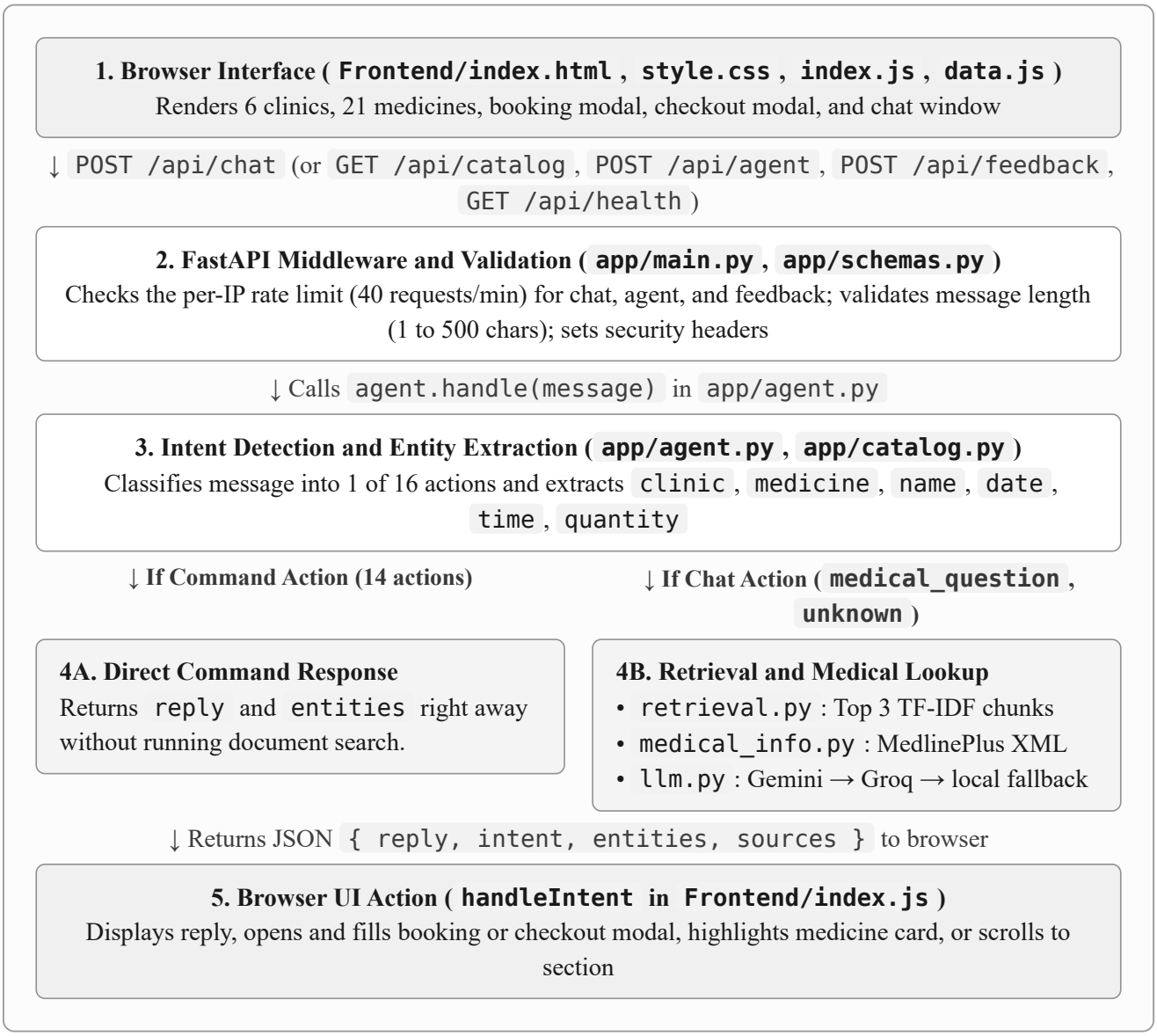


Figure 2: System architecture and end-to-end request flow.

2.1. API Endpoints

The server provides five routes in `app/main.py`:

- `GET /api/health`: Returns `"ok"`, the active LLM provider (`"none"` by default, or `"gemini+groq"` , `"gemini"` , `"groq"`), and the number of indexed knowledge chunks (`14`).
- `GET /api/catalog`: Returns the 6 clinics and 21 medicines from `app/catalog.py`.
- `POST /api/chat`: Runs intent detection and entity extraction, then either returns a command response or runs local search, MedlinePlus lookup, and answer formatting.
- `POST /api/agent`: Runs `agent.handle(command)` and returns `{ action, entities, reply }` directly without document search.
- `POST /api/feedback`: Appends `{ timestamp, message, predicted_action, correct_action }` to a local JSON Lines file (`data/feedback.jsonl`).

3. Intent Detection and Entity Extraction

3.1. Rule Evaluation Order

To prevent overlapping keywords from triggering the wrong action, `app/agent.py` checks rules from top to bottom in this exact order:

1. **Emergency check:** Matches phrases such as `chest pain`, `stroke`, `cannot breathe`, `unconscious`, `severe bleeding`, `suicide`, `overdose`, `seizure`, or `choking`.
2. **Small talk:** Matches short greetings (`hello`, `hi`, `salam` up to 4 words), `thanks`, and `goodbye`.
3. **Help:** Matches `help`, `what can you do`, `how does this work`, `rules`, `capabilities`, or `menu`.
4. **Medicine + dosage terms:** Matches a medicine name with `dosage`, `dose`, `mg`, `strength`, or `how many`.
5. **Medicine + price terms:** Matches a medicine name with `price`, `cost`, or `how much`.
6. **Medicine + side effects or interactions:** Classifies as `medical_question` so the reply includes a medical disclaimer.
7. **Medicine + info terms:** Matches a medicine name with `what is`, `tell me about`, `do you have`, `info`, or `used for`.
8. **Medicine + buy terms:** Matches a medicine name with `buy`, `order`, `purchase`, `need`, or `want`.
9. **Clinic + booking terms:** Matches a clinic with `book`, `appointment`, `schedule`, `reserve`, `visit`, `need`, `want`, or `see`.
10. **Generic booking or buying:** Matches booking or buying words when no clinic or medicine is named, and asks the user to specify one.
11. **Section navigation:** Matches keywords for `contact`, `pharmacy`, `clinics`, or `home`.
12. **Single entity mention:** A clinic name alone triggers `book_appointment`; a medicine name alone triggers `medicine_info`.
13. **Medical terms or fallback:** Matches general health terms (`diabetes`, `flu`, `fever`, `symptoms`, `treatment`, etc.) as `medical_question`, or returns `unknown`.

Action	Type	Example Input	Resulting Behavior
`book_appointment`	Command	book a dentist for Ahmed tomorrow at 5 pm	Opens booking modal with fields filled (or scrolls to `#clinics` if no clinic is named)
`buy_medicine`	Command	buy two packs of betadine for Ali	Opens checkout modal with quantity and name (or scrolls to `#pharmacy` if no medicine is named)
`price`	Command	how much is Aspirin?	Replies with price (78 EGP) and highlights card
`dosage`	Command	what is the dosage of augmentin?	Replies with strength (1 g) and highlights card
`medicine_info`	Command	tell me about augmentin	Replies with use, price, strength, prescription note, and highlights card
`navigate_clinics`	Command	show me the clinics	Scrolls page to `#clinics`
`navigate_pharmacy`	Command	open the pharmacy	Scrolls page to `#pharmacy`
`navigate_contact`	Command	what is your phone number?	Replies with contact details and scrolls to `#contact`
`navigate_home`	Command	open home	Scrolls page to `#home`
`greeting`	Command	hello	Returns welcome message
`help`	Command	what can you do	Lists supported commands and examples
`thanks`	Command	thanks	Returns polite reply
`goodbye`	Command	goodbye	Returns closing message
`emergency`	Command	I have chest pain	Shows warning message directing the user to local emergency services
`medical_question`	Chat	what is diabetes?	Fetches MedlinePlus summary, source link, and medical disclaimer
`unknown`	Chat	do you deliver?	Searches local `docs.md` chunks (or MedlinePlus) and returns answer

Table 1: All 16 actions handled by `app/agent.py`.

3.2. Entity Extraction Rules

Entity	Function	How It Works
`clinic`	`find_clinic`	Matches clinic keywords (singular and plural); maps `"doctor"` to `General Medicine`.
`medicine`	`find_medicine`	Checks exact aliases (longest first), then `SequenceMatcher` similarity ratio ≥ 0.82 for typos.
`name`	`find_name`	Extracts up to 3 words after `my name is`, `name is`, or `for <Name>`, stopping at stopwords.
`date`	`find_date`	Parses `today`, `tomorrow`, `day after tomorrow`, weekday names, `YYYY-MM-DD`, `DD/MM/YYYY`, and month names into `YYYY-MM-DD`.
`time`	`find_time`	Converts 12-hour (`5 pm`, `10:30 am`) and 24-hour (`17:00`) times into `HH:MM`.
`quantity`	`find_quantity`	Reads numbers before `packs/boxes/units/bottles`, after `buy/order`, or number words `one` to `ten` (clamped between 1 and 20).

Table 2: Entity extraction functions in `app/agent.py`.

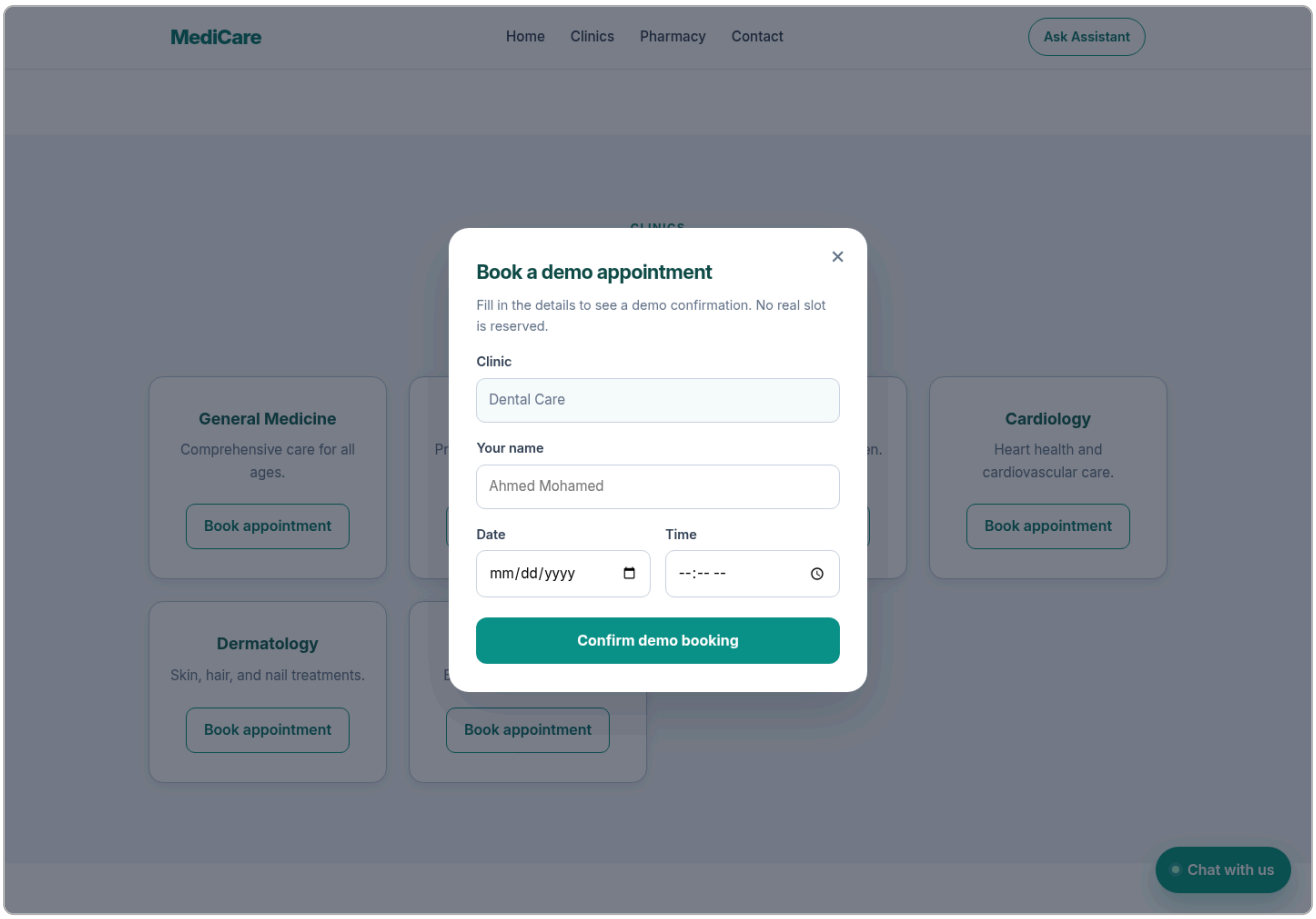


Figure 3: Clinics section and the appointment booking modal opened for Dental Care.

4. Catalogue, Retrieval, and Medical Grounding

4.1. Clinic and Medicine Data

The catalogue is stored in `app/catalog.py` and mirrored in `Frontend/data.js`:

- **6 Clinics:** General Medicine, Dental Care, Pediatrics, Cardiology, Dermatology, and Orthopedics.
- **21 Medicines:** Each frontend card displays a local product photo, the medicine name, the price in EGP (for example, Panadol Extra at 45 EGP, Aspirin at 78 EGP, Telfast at 63 EGP, Metformin 500mg at 8 EGP, and Betadine at 30 EGP), and a "Buy now" button. The backend catalogue also stores the strength, short description, aliases, and whether a prescription is required (7 prescription medicines and 14 over-the-counter medicines).

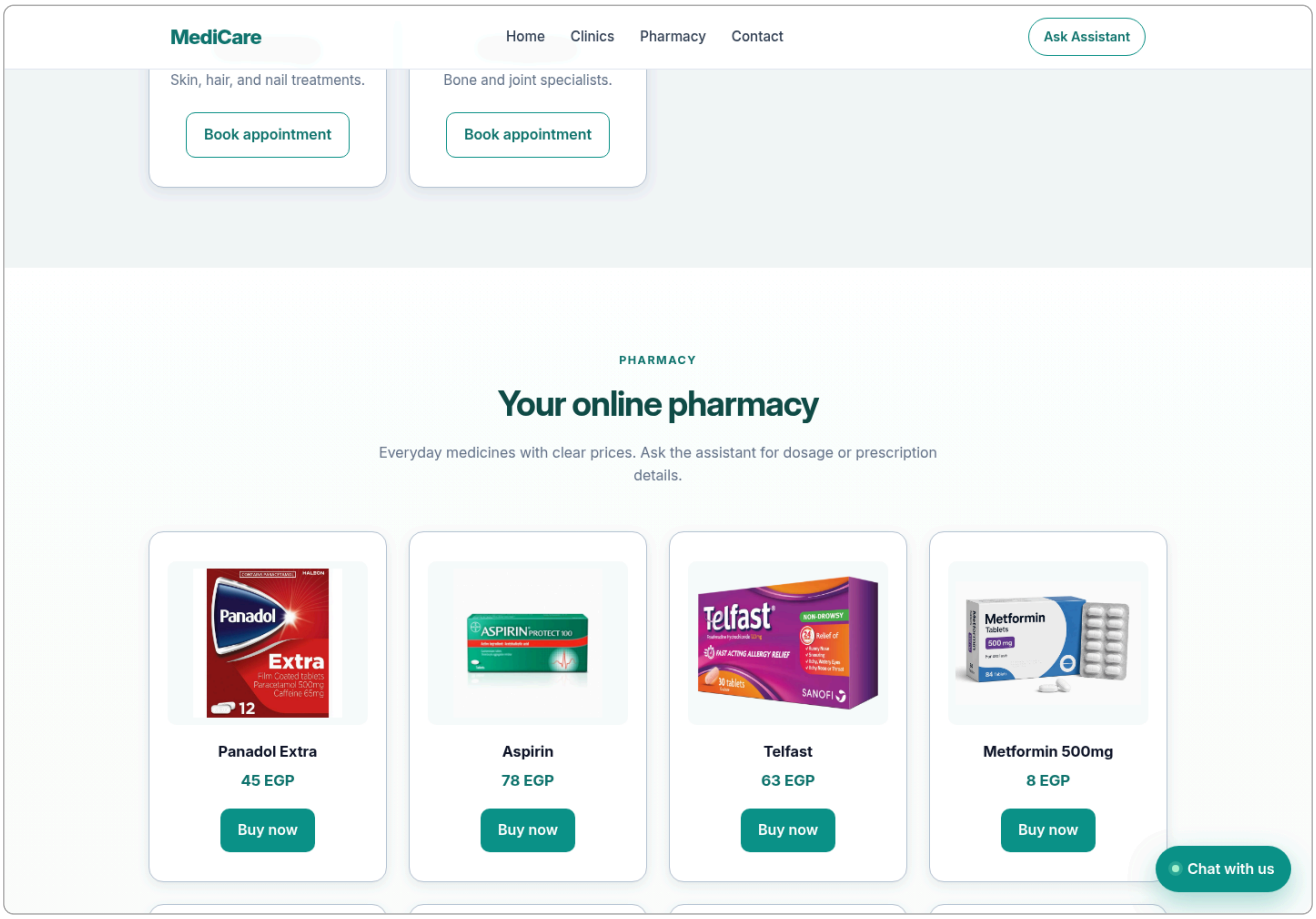


Figure 4: Online pharmacy section showing the medicine cards (including Aspirin at 78 EGP and Metformin 500mg).

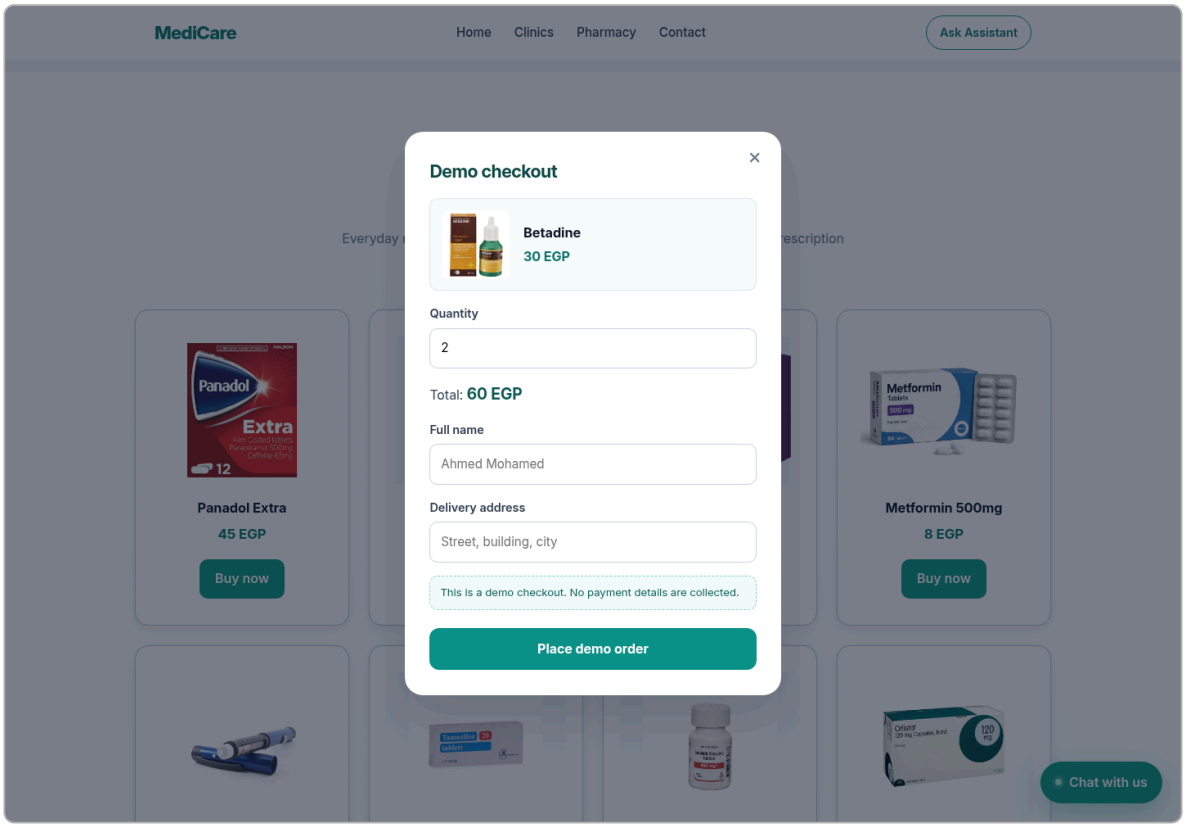


Figure 5: Demo pharmacy checkout modal calculating the total price for multiple packs.

4.2. Local TF-IDF Retrieval and MedlinePlus Lookup

The local knowledge base (`app/knowledge/docs.md`) covers clinics, booking rules, pharmacy prices, ordering and delivery rules, contact details, and common questions.

1. **Local TF-IDF search (`app/retrieval.py`):** At server startup, `chunk_markdown` splits `docs.md` by Markdown headings into 14 chunks that target about 600 characters each; a single oversized paragraph stays intact. `RetrievalIndex` fits a scikit-learn `TfidfVectorizer` (`stop_words="english"` , `ngram_range=(1, 2)` , `sublinear_tf=True`) on the 14 chunks. At query time, `search` computes cosine similarity between the query and the chunk matrix and returns the top 3 chunks with a positive score.
2. **MedlinePlus XML lookup (`app/medical_info.py`):** When a message is classified as `medical_question` (or `unknown` with no matching local chunks) and `MEDLINEPLUS_ENABLED` is `true` , `fetch_medlineplus` removes stopwords from the query and sends an HTTP `GET` request to `https://wsearch.nlm.nih.gov/ws/query` (`db=healthTopics` , `retMax=3`). It parses the XML response, strips HTML tags and entities, trims the summary to at most 1,400 characters, and returns the title, summary, and source URL.
3. **Answer generation and fallback (`app/llm.py`):** When `GEMINI_API_KEY` and `GROQ_API_KEY` are configured (`LLM_PROVIDER=auto`), `generate` first calls Gemini (`gemini-3.5-flash-lite`). If Gemini hits its daily quota (marked exhausted for the day) or fails, it automatically switches to Groq (`openai/gpt-oss-120b`). If Groq also fails or no API keys are set (`none`), `fallback_answer` formats the response directly from the MedlinePlus summary (with the source link returned in `sources` and the medical disclaimer) or from the top local `docs.md` chunk.

5. Security, Testing, and Deployment

Security Control	Implementation
Output escaping	Chat messages are inserted with `textContent`; clinic and medicine fields use `escapeHtml`.
Input validation	Pydantic models trim whitespace and enforce 1 to 500 characters on all chat and agent inputs.
Rate limiting	`RateLimiter` in `app/main.py` allows 40 requests per 60-second sliding window per client IP on chat, agent, and feedback; health and catalogue stay exempt.
Security headers	Every response sets `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy`, and `Permissions-Policy`.
Content Security Policy	Allows only same-origin scripts, images, and connections (plus Google Fonts, and CDN assets on `/docs`).
No payment data	The demo checkout asks only for a name and delivery address; no card, CVV, or expiry fields exist.

Table 3: Security controls implemented in the application.

Test File	Tests	What It Verifies
`test_agent.py`	79	Intent classification, entity extraction, date and time parsing, fuzzy medicine matching, emergency replies
`test_api.py`	26	Endpoints, validation, security headers, rate-limit scope, feedback logging, static files, frontend mount
`test_llm.py`	22	Gemini → Groq → local fallback chain, daily-quota skipping, error handling, fallback formatting
`test_retrieval.py`	10	Markdown chunking, oversized blocks, TF-IDF ranking, empty index handling, context limits
`test_medical_info.py`	11	MedlinePlus XML parsing, snippets, truncation, HTTP/XML errors, oversized responses, query cleaning
`test_data.py`	8	Parity between `Frontend/data.js` and `app/catalog.py`, catalogue invariants, local assets, `docs.md` coverage
`test_schemas.py`	7	Whitespace normalisation and request validation

Table 4: Automated test suite (`163 passed` with `pytest`).

The application is deployed on Render.

6. Limitations

- The assistant provides general health information only and does not diagnose conditions or prescribe medicines.
- Appointment bookings and pharmacy checkout orders are demo workflows and are not stored in a database or connected to a payment gateway.
- Local search uses TF-IDF keyword matching, so synonyms that are not listed in `docs.md` or the alias table rely on the MedlinePlus lookup or return the fallback message.